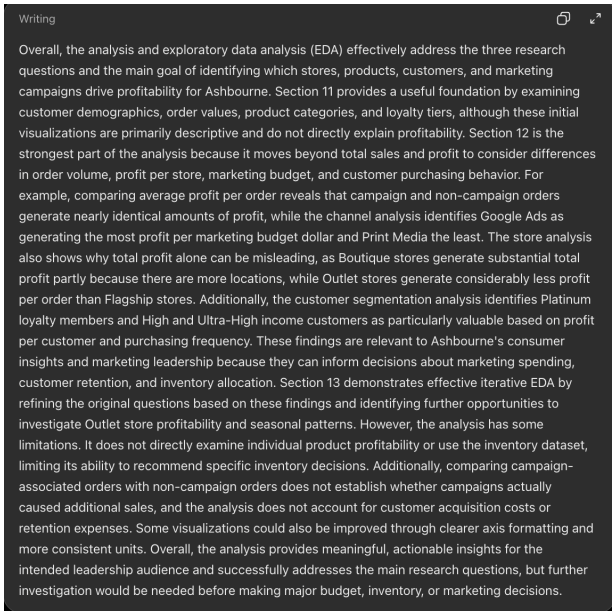
# BTA 522 Generative AI Appendix

ChatGPT was used for the first 2 prompts and Claude was used for the rest.

**ChatGPT Prompt (Number 14)**: In my attached Google Colab file, write a paragraph fully evaluating my analysis and EDA sections (#11-#14) based on how well they answer the goals/parameters/target audience mentioned in question #1. Justify your answers and explain.



Writing

Overall, the analysis and exploratory data analysis (EDA) effectively address the three research questions and the main goal of identifying which stores, products, customers, and marketing campaigns drive profitability for Ashbourne. Section 11 provides a useful foundation by examining customer demographics, order values, product categories, and loyalty tiers, although these initial visualizations are primarily descriptive and do not directly explain profitability. Section 12 is the strongest part of the analysis because it moves beyond total sales and profit to consider differences in order volume, profit per store, marketing budget, and customer purchasing behavior. For example, comparing average profit per order reveals that campaign and non-campaign orders generate nearly identical amounts of profit, while the channel analysis identifies Google Ads as generating the most profit per marketing budget dollar and Print Media the least. The store analysis also shows why total profit alone can be misleading, as Boutique stores generate substantial total profit partly because there are more locations, while Outlet stores generate considerably less profit per order than Flagship stores. Additionally, the customer segmentation analysis identifies Platinum loyalty members and High and Ultra-High income customers as particularly valuable based on profit per customer and purchasing frequency. These findings are relevant to Ashbourne's consumer insights and marketing leadership because they can inform decisions about marketing spending, customer retention, and inventory allocation. Section 13 demonstrates effective iterative EDA by refining the original questions based on these findings and identifying further opportunities to investigate Outlet store profitability and seasonal patterns. However, the analysis has some limitations. It does not directly examine individual product profitability or use the inventory dataset, limiting its ability to recommend specific inventory decisions. Additionally, comparing campaign-associated orders with non-campaign orders does not establish whether campaigns actually caused additional sales, and the analysis does not account for customer acquisition costs or retention expenses. Some visualizations could also be improved through clearer axis formatting and more consistent units. Overall, the analysis provides meaningful, actionable insights for the intended leadership audience and successfully addresses the main research questions, but further investigation would be needed before making major budget, inventory, or marketing decisions.

**ChatGPTPrompt:** What does this warning mean:

/tmp/ipykernel_6628/2112296920.py:1: FutureWarning: The default of observed=False is deprecated and will be changed to True in a future version of pandas. Pass observed=False to retain current behavior or observed=True to adopt the future default and silence this warning.

```
  campaign_summary = merged_df.groupby("campaign_status").agg(
campaign_summary = merged_df.groupby("campaign_status").agg(
    total_sales=("net_sales", "sum"),
    profit=("profit", "sum")
).round(2)
Campaign_summary
```

This is a **FutureWarning**, not an error. Your code is still working correctly.

Pandas is warning you that the default behavior of `.groupby()` will change in a future version.

**1. What does `observed=False` mean?**

Because you converted `campaign_status` to the `category` data type earlier, Pandas has a special setting for grouping categorical variables.

- `observed=False` — Includes all defined categories, even if some have no observations.
- `observed=True` — Includes only categories that actually appear in your data.

For your dataset, you have two categories: `"Campaign"` and `"No Campaign"`.

Since both appear in your data, **the results should be identical with either setting**.

**2. How to fix the warning**

Simply add `observed=True` inside `.groupby()`:

Python Run

```
campaign_summary = merged_df.groupby(
    "campaign_status",
    observed=True
).agg(
    total_sales=("net_sales", "sum"),
    profit=("profit", "sum")
).round(2)

campaign_summary
```

**3. Will this affect your results?**

For your current analysis, no, assuming both campaign statuses are represented.

The difference would only matter if you had a defined category with no observations.

For example, if `"No Campaign"` existed as a category but there were no orders without campaigns:

- `observed=False` would still include `"No Campaign"` in the grouped results.
- `observed=True` would exclude it.

**My recommendation:** Use `observed=True` for your Ashbourne project because you're interested in analyzing categories that actually have observations. It also removes the warning and aligns with Pandas' upcoming default behavior.

**Prompt:** What is the different between using sns vs .plot.bar

Both make bar charts. The difference is where the data comes from and who does the calculating.

| | `sns.barplot` | `.plot.bar()` |
|---|---|---|
| Starts from | A table of raw rows, such as `merged_df` | A table you've already summarized, such as `bracket_summary` |
| Does the totaling | Yes, with `estimator="sum"` (the default is the mean) | No, it plots the numbers as they are |
| Names the variables | Yes: `x="income_bracket"`, `y="profit"` | No, the row labels become the x-axis and the columns become the bars |
| Two measures side by side | Needs `.melt()` and `hue=` | Select both columns and it pairs them automatically |
| Error bars | Drawn by default (use `errorbar=None`) | None |
| Bar order | Set with `order=` | Follows the table's row order |
| Look | Seaborn styling and legend | Basic matplotlib styling |

In practice:

- `sns.barplot` suits charts straight from `merged_df`, where you want seaborn to do the grouping, as in your store type chart.
- `.plot.bar()` suits charts from a summary table you've already built and displayed. It's shorter, and the chart exactly matches the table on screen, which makes it easy to check.

**Prompt:** When I subset my 6 files into dataframes that hold the variables of interest for my analysis and cleaning, would you recommend creating a copy of the original dataframe for these. Explain and justify your suggestion

Yes, I'd recommend working on copies. Here is why, and when it matters most.

**Why a copy is worth it**

1. **Protecting the raw data.** Your cleaning steps (dropping rows, imputing nulls, recoding categories, type conversions) will change the data. If you work directly on the loaded frame, you lose the ability to compare "before" and "after" without re-reading the CSV. With a pristine original in memory, you can always check what a cleaning step actually changed (e.g., how many `rating` nulls you imputed, how many `campaign_id` NaNs you filled).
2. **Avoiding pandas' view-vs-copy ambiguity.** When you subset with `df[cols]` or a boolean filter, pandas may return a view or a copy depending on internal details. If you then assign to the subset, you can get `SettingWithCopyWarning`, or worse, silently modify the original, or silently *fail* to modify what you intended. Calling `.copy()` makes the subset an independent object and removes that ambiguity entirely.
3. **Reproducibility and auditability.** Keeping the originals untouched makes your cleaning pipeline easy to defend: raw → subset copy → cleaned. That matters if you need to document data-quality decisions or rerun the analysis later.
4. **Safe experimentation.** You can try an aggressive cleaning approach (say, removing outlier orders or a different imputation method) on one copy, and revert by re-copying from the original instead of re-reading files.

**Prompt:** Now that I've created dataframes for each of my files, how would you suggest to show them all in one table  to describe data types and missing values



Building a per-column summary across dataframes ›

The cleanest approach is a long-format summary table: one row per column, with a column identifying which dataframe it came from. Stacking all 6 files this way gives you a single table (~57 rows for your data) that is easy to scan, sort, filter, and export.

python

```python
import pandas as pd

# Adjust the names to match your own dataframes
dfs = {
    "stores": stores,
    "products": products,
    "orders": orders,
    "campaigns": campaigns,
    "inventory": inventory,
    "customers": customers,
}

def summarize(name, df):
    return pd.DataFrame({
        "dataframe": name,
        "column": df.columns,
        "dtype": df.dtypes.astype(str).values,
        "non_null": df.notna().sum().values,
        "missing": df.isna().sum().values,
        "missing_pct": (df.isna().mean() * 100).round(2).values,
        "unique": df.nunique().values,
    })

summary = pd.concat([summarize(n, d) for n, d in dfs.items()], ignore_index=True)

summary  # or summary.to_csv("data_quality_summary.csv", index=False)
```

**Why this structure**

- **One row per variable** keeps the table readable, whereas a wide layout with 6 side-by-side blocks would be awkward because the files have different columns.
- `missing` **and** `missing_pct` **together** matter: a count of 40 is trivial in Orders (26,500 rows) but significant in Stores (16 rows).
- `unique` is a cheap extra that helps you spot IDs, low-cardinality categoricals, and constant columns.
- **Sorting by** `missing_pct` puts the problem columns on top:

**Prompt:** Why is it beneficial to turn the order date from object to datetime?

Because as text, Python treats `order_date` as a string like "2023-01-01" and has no idea it's a date. Once it's a datetime, Python can do date math and date-based grouping.

What a datetime lets you do

- **Pull out parts of the date.** `orders_voi["order_date"].dt.year` gives the year, and `.dt.month_name()` gives the month. With text, those commands don't exist. This is what lets you compare profit by year or season.
- **Subtract dates.** `delivery_date - order_date` gives the number of days between them. With text, subtracting two strings gives an error.
- **Sort in the correct order.** Dates formatted as year-month-day happen to sort correctly even as text, but other formats (like `1/2/2023`) don't. A real date always sorts correctly.
- **Filter by date.** `orders_voi[orders_voi["order_date"] >= "2025-01-01"]` finds orders from 2025 onward by meaning, not by comparing characters.
- **Plot over time.** Charts put dates on the axis in order and spacing, instead of treating each date as a separate label.
- **Match its conceptual type.** `order_date` is a date, so storing it as one fixes the mismatch we noted in Step 4.

A small example

```python
# As text
"2023-01-01" - "2023-01-05"     # error: can't subtract strings

# As dates
pd.Timestamp("2023-01-05") - pd.Timestamp("2023-01-01")    # 4 days
```

For your project, the biggest benefit is that you can group orders by year or month, and compare campaign periods to the orders that fall inside them.

**Prompt:** is there a way to combine these two into one chart
sns.countplot(data=stores_voi, x="store_type", color="steelblue")
plt.show()
sns.countplot(data=products_voi, x="category", color="steelblue")
plt.show()

Yes. Make a grid of panels with `plt.subplots`, then draw each chart into its own panel with `ax=`. You write one line per chart, with no loop and no placeholder name:

```python
fig, axes = plt.subplots(2, 2, figsize=(12, 8))

sns.countplot(data=stores_voi,    x="store_type",    ax=axes[0, 0], color="steelblue"
sns.countplot(data=products_voi,  x="category",      ax=axes[0, 1], color="steelblue"
sns.countplot(data=campaigns_voi, x="channel",       ax=axes[1, 0], color="steelblue"
sns.countplot(data=orders_voi,    x="return_status", ax=axes[1, 1], color="steelblue"

plt.tight_layout()
plt.show()
```

How it works

- `plt.subplots(2, 2, figsize=(12, 8))` creates one figure with a 2-by-2 grid of panels and returns the panels in `axes`.
- `axes[0, 0]` means row 0, column 0 (top left). `axes[0, 1]` is top right, `axes[1, 0]` is bottom left, and `axes[1, 1]` is bottom right.
- `ax=axes[0, 0]` tells `countplot` to draw in that panel instead of making a new chart.
- `plt.show()` appears once at the end, so all four charts display together.

**Prompt:** Help me determine which variables of interest would be best to visualize in a bar char to show “clean” data and explain why you chose them

I picked them with one rule: **text variables whose labels could be misspelled or inconsistent, and which your analysis will group or filter by.** That left 11 variables.

Why these are included

| Variable | Why it's in |
|---|---|
| gender, income_bracket, loyalty_tier, acquisition_source | Customer segments for Question 3, or likely to be used for them |
| channel | The core variable in Question 2 |
| return_status | Needed for the returns part of Question 1 |
| category, sub_category | Product profit in Question 1 |
| store_type | Store comparisons in Question 1 (Flagship vs. Boutique vs. Outlet vs. E-Commerce) |
| season, material | Product attributes, included for completeness |

Why others are left out

- **ID columns** (`customer_id`, `order_id`, etc.): they are numbers, not spelled labels, so there is nothing to misspell.
- **Dates and numeric columns:** these have no spelling or case problem.
- `city`, `campaign_name`, `product_name`, `store_name`: these have too many distinct values (15 to 41) to read as a bar chart. I did check them for stray spaces and case differences in my first scan of the data, and they were clean, but they aren't in the Step 5 code.
- `rating`: it is a number from 1 to 5, so it can't have spelling variants. I plotted it in an earlier version, but it isn't needed for the consistency check.

**Prompt:** In my data how do you know which variables  would be best to turn to categorical data type

Convert a column when it holds a **small, fixed set of repeated labels** and arithmetic on it would be meaningless. The `unique` column from your summary table lets you test this quickly.

**Decision criteria**

1. **Low cardinality relative to row count.** A rough rule is that unique values are well under ~5% of rows, or at most a few dozen. `store_type` has a handful of values across 16 rows, and `return_status` has a few across 26,500.
2. **The values are labels, not quantities.** If averaging or summing the column makes no sense, it's a candidate.
3. **There is a natural order.** Columns like `income_bracket` and `loyalty_tier` should become *ordered* categoricals so sorting, comparisons, and plots respect the sequence (Bronze < Silver < Gold, not alphabetical).
4. **It repeats enough to benefit.** You get lower memory use, faster `groupby`, consistent plot ordering, and a built-in guard against typos creeping in.

**Applied to your files**

| File | Convert | Leave alone |
|---|---|---|
| Stores | `city`, `store_type` | `store_name` (one per store), `square_footage`, `open_date` |
| Products | `category`, `sub_category`, `season`, `material` | `product_name` (unique per row), prices and costs |
| Orders | `return_status` | `quantity`, `discount_pct`, the money columns, dates |
| Campaigns | `channel` | `campaign_name` (unique per row), `budget` |
| Customers | `gender`, `acquisition_source`, `city`, `income_bracket` (ordered), `loyalty_tier` (ordered) | `age`, `join_date` |
| Inventory | none | `stock_level`, `last_restock_date` |

**Judgment calls**

- **ID columns** (`customer_id`, `product_id`, `store_id`, `campaign_id`, etc.) are numbers that behave like labels, but I'd keep them as integers. You'll use them for merges, and mismatched dtypes between the two sides of a join cause trouble. Cardinality is also high (3,000 customers), so there's little payoff. `campaign_id` is a float only because of NaNs, so if you want a clean integer, use the nullable `Int64`.
- `rating` is the genuine borderline case. It's a 1-5 integer, which is ordinal, but if you want mean rating or correlations, keep it numeric. Convert it to an ordered categorical only if you plan to treat it purely as a distribution of levels.
- `discount_pct` may have only a few distinct values, but it's a quantity you'll compute with, so keep it numeric.
- **Free-text or near-unique columns** (`product_name`, `store_name`, `campaign_name`) gain nothing from categorical type.

**Prompt:** Are there any variable transformations that would be worth doing for a datetime variable?

Yes. The first step is converting to a true datetime type, and after that you can derive features that make the dates analytically useful.

**Step 1: Convert to datetime**

Your date columns will load as `object` strings. Convert them first, with `errors="coerce"` so malformed entries become `NaT` instead of crashing, then compare null counts before and after to catch bad values.

```python
orders["order_date"] = pd.to_datetime(orders["order_date"], errors="coerce")
orders["delivery_date"] = pd.to_datetime(orders["delivery_date"], errors="coerce")
```

**Step 2: Extract calendar components**

These let you group and plot by period, which is usually the main reason to transform dates.

```python
orders["order_year"] = orders["order_date"].dt.year
orders["order_month"] = orders["order_date"].dt.month
orders["order_quarter"] = orders["order_date"].dt.quarter
orders["order_dow"] = orders["order_date"].dt.day_name()
orders["is_weekend"] = orders["order_date"].dt.dayofweek >= 5
orders["order_ym"] = orders["order_date"].dt.to_period("M")  # handy for monthly trend
```

Month and quarter are particularly relevant for you because Products has a `season` column, so you can test whether seasonal products sell in their intended season. Day of week is useful for spotting weekend versus weekday purchasing patterns.

**Prompt:** What is a feather file and how to start writing one for my data?

A Feather file is a binary file format for saving a pandas table (DataFrame) to disk.

**Why it's useful**

- **It's fast.** Feather files read and write much faster than CSV, because the data is stored in a form that is close to how it sits in memory.
- **It keeps your data types.** A CSV forgets types: your `category` columns, datetimes and nullable integers all come back as plain text or floats. Feather saves each column's type and restores it when you read the file back, which is why the assignment has you save the cleaned data and reload it.

**How to write one**

```python
import os
os.makedirs("data", exist_ok=True)              # make the data folder if it doesn't

orders_voi.to_feather("data/orders_voi.feather")  # write the DataFrame to a file
orders_clean = pd.read_feather("data/orders_voi.feather")   # read it back into pandas

print(orders_clean.dtypes)                       # check the types survived
```

- `.to_feather("path")` is a DataFrame method that saves the table.
- `pd.read_feather("path")` loads it back into a new DataFrame.
- It needs the `pyarrow` package, which Colab already has.
- Feather requires the table to have the default 0, 1, 2... row index. If you get an error about a non-default index, add `.reset_index(drop=True)` before saving.
- In Colab, the `data` folder lives in the temporary session, so it disappears when the session ends. Your earlier cells rebuild it each time you use Run all.

**Prompt:** What does import os and os.makedirs do

`import os` loads Python's built-in `os` module, a toolkit for working with the operating system: files, folders and paths. You have to import it before you can use anything inside it, like `import pandas as pd` for pandas. It's part of Python itself, so there's nothing to install.

`os.makedirs("data", exist_ok=True)` creates a folder named `data` in your current working directory.

- `"data"` is the name of the folder to create. You can also give a nested path like `"data/raw"`, and `makedirs` will create every folder along the way.
- `exist_ok=True` means "don't raise an error if the folder already exists." Without it, running the cell a second time fails with a `FileExistsError`, because the folder was made the first time. With it, the call does nothing if the folder is already there, so your notebook works with Run all.

You need this line before saving because `to_feather("data/orders_voi.feather")` can write a file into a folder only if the folder exists. If it doesn't, you get an error like "No such file or directory". `makedirs` creates the folder first, so the save works.

**Prompt:** How do I change the x labels so they are horizontal

Add `rot=0` to `plot.bar()`. It sets the rotation of the x-axis labels, and 0 makes them horizontal:

```python
campaign_summary[["net_sales", "profit"]].plot.bar(rot=0)
```

Do the same on the second chart:

```python
campaign_summary[["avg_net_sales", "avg_profit"]].plot.bar(rot=0)
```

Another way is to add `plt.xticks(rotation=0)` before `plt.show()` in each cell. Both do the same thing, so use whichever is easier for you to remember.

Your first comment still says "so I compare the average profit for each instead." You don't need it anymore, since the comment below the second chart covers the same point.